

В данной работе реализован программный код на языке Python, демонстрирующий применение шаблона проектирования «Адаптер» (**Adapter**).

Цель программы — показать, как можно расширить функциональность уже существующего класса, не изменяя его исходный код. В данном случае имеется класс **Office**, который содержит информацию об офисе (название и адрес), но не имеет возможности изменять адрес (переезд офиса).

Согласно условию задания, изменять существующий класс нельзя, поэтому используется паттерн «Адаптер», который позволяет добавить новый функционал через дополнительный класс.

Что делает данный код

Программа моделирует работу офиса компании. В начале создается объект офиса с определенным названием и адресом. Далее с помощью адаптера добавляется возможность изменить адрес офиса, то есть реализуется функция «переезда».

Таким образом, программа выполняет следующие действия:

1. Создает объект офиса с начальным адресом;
2. Выводит информацию об офисе;
3. Выполняет «переезд» офиса на новый адрес;
4. Снова выводит обновленную информацию.

Структура программы

1. Класс Office

Класс **Office** является существующим классом, который нельзя изменять. Он содержит:

- название офиса;
- адрес;
- метод **get_info()**, который выводит информацию.

Этот класс представляет собой так называемый **Adaptee** — объект, который необходимо адаптировать

2. Интерфейс IMovable

Интерфейс **IMovable** определяет новый функционал — возможность перемещения офиса. В нем объявлен метод:

- **move(new_address)**

Этот интерфейс представляет собой **Target**, то есть тот интерфейс, который ожидает клиент.

3. Класс OfficeAdapter

Класс `OfficeAdapter` является ключевым элементом программы. Он:

- наследуется от класса `Office`;
- реализует интерфейс `IMovable`.

В этом классе реализован метод `move()`, который изменяет адрес офиса.

Таким образом, `OfficeAdapter` выполняет роль **Adapter** — он “оборачивает” существующий класс и добавляет новый функционал без изменения исходного кода.

Для чего нужен этот код

Данный код нужен для демонстрации того, как можно:

- расширять функциональность существующих классов;
- не изменять исходный код (что важно в реальных проектах);
- объединять несовместимые интерфейсы;
- делать код более гибким и масштабируемым.

Паттерн «Адаптер» особенно полезен в следующих случаях:

- при работе со сторонними библиотеками;
- при интеграции старого и нового кода;
- когда нельзя изменить существующий класс;
- при необходимости добавить новый функционал без нарушения архитектуры.

Заключение

В результате выполнения задания был реализован паттерн «Адаптер», позволяющий расширить возможности существующего класса без его изменения.

Программа наглядно демонстрирует, как можно добавить новый функционал (переезд офиса), используя дополнительный класс-адаптер. Это делает систему более гибкой и удобной для дальнейшего расширения.

Полученные знания могут быть применены при разработке реальных программных систем, особенно при интеграции различных компонентов с несовместимыми интерфейсами.

```
from abc import ABC, abstractmethod
```

```

# =====
# Существующий класс (его менять нельзя)
# =====
class Office:
    def __init__(self, name: str, address: str):
        self.name = name
        self.address = address

    def get_info(self):
        return f"Офис: {self.name}, Адрес: {self.address}"

# =====
# Интерфейс (Target)
# =====
class IMovable(ABC):
    @abstractmethod
    def move(self, new_address: str):
        pass

# =====
# Адаптер (Adapter)
# =====
class OfficeAdapter(Office, IMovable):
    def move(self, new_address: str):
        print(f"\nПеремещение офиса '{self.name}'...")
        self.address = new_address
        print(f"Новый адрес: {self.address}")

# =====
# Клиентский код
# =====
if __name__ == "__main__":
    # создаем обычный офис
    office = OfficeAdapter("Norbit", "Алматы, Абая 10")

    print("До переезда:")
    print(office.get_info())

    # используем адаптер (новый функционал)
    office.move("Алматы, Назарбаева 50")

    print("\nПосле переезда:")
    print(office.get_info())

```

До переезда:

Офис: Norbit, Адрес: Алматы, Абая 10

Перемещение офиса 'Norbit'...

Новый адрес: Алматы, Назарбаева 50

После переезда:

Офис: Norbit, Адрес: Алматы, Назарбаева 50